

STATE

LLMsVerifier nezha wiring — HONEST-BLOCKED (operator decision) · 2026-06-16T20:31Z

Revision: 1 Last modified: 2026-06-16T20:35:00Z

Verdict (autonomous, §11.4.101 safest/zero-risk decision)

LLMsVerifier wiring NOT performed. helixtranslate left on the working direct path (NON-degraded). LLMSVERIFIER_ENABLED unset, verifier /api/models count:0 unchanged, NO config/image/network touched. The never-degrade guard held: never enable until verifier count>0.

§11.4.142 INDEPENDENT review of the seed feature (submodule commit 6e6b0309) — GO

Independently reviewed (read-only agent, evidence in INDEPENDENT_ANALYSIS_6e6b0309.md): the seed extension is CORRECT, idempotent, decoupled, non-fatal, and tested:

- seed.FromConfig(cfg, db) called in runServer() after DB init, non-fatal; seed.go for i := range cfg.LLMs idempotently upserts providers+models; reads only config.Config (decoupled).
- 6 tests in seed_test.go use a real in-memory SQLite DB incl. a RED-baseline asserting db.ListModels (the exact call /api/models uses) returns the seeded rows. The feature catches its own negation.
- **GO** — the code is sound. The blocker is purely deployment-side (below), not the seed code.

Why the wiring is blocked (FACT, §11.4.6 — captured this session)

1. **Live verifier image predates the seed code.** Running `llm-verifier:nezha` was built **2026-06-16 15:46:19 UTC**; the seed commit `6e6b0309` landed **2026-06-16 19:26:59 UTC** (~3.5h later). So the live verifier has NO seed-at-boot; `/api/models` → `{"count":0,"models":[]}`.
2. **No reproducible on-nezha rebuild path.** `docker-compose.app.yml` runs the image built from "Dockerfile.nezha", but the verifier dir on nezha has NO build context/source, and the submodule carries only a generic `llms_verifier/Dockerfile` (no `Dockerfile.nezha`). The nezha build invocation that produced `llm-verifier:nezha` is external/unknown — rebuilding an encrypted-DB cgo service blind risks a broken image.
3. **config.yaml is operator-owned + secret-bearing + Permission-denied.** Owned by container-user 165531, perm 600, bind-mounted `:ro,U`. It holds the substituted real `JWT_SECRET + DATABASE_ENCRYPTION_KEY`. `llms:` is `[]`. Populating it would require handling those secrets blind (§11.4.10) with no secret-safe render script present. NOT mutated (§11.4.10/§11.4.122/§9.2).
4. **Even if 1–3 were solved, seeded models are VerificationStatus: "pending"**, but the helixtranslate gate requires "verified" (`OverallScore > MinScoreThreshold`) — so a real verification pass (`POST /api/models/{id}/verify`, real provider calls) would still be required before `count(verified)>0`.

Never-degrade proof (§11.4.69)

helixtranslate translation path stays in-process (bridge `LLMSVERIFIER_API_URL` unset) → direct DeepSeek/Groq path intact. `GET /api/v1/verified-models` returns the graceful "integration disabled" state. No regression.

§11.4.66 — operator options (for the morning)

- **Option A (recommended):** operator rebuilds `llm-verifier:nezha` from the seed-bearing submodule (`6e6b0309`) via the original nezha build path; operator populates the verifier `config.yaml llms:` with `deepseek+groq` (handling its secrets); restart → seed at boot → `POST /api/models/{id}/verify` to reach verified; confirm `/api/models count>0`; THEN wire helixtranslate (network connect + `LLMSVERIFIER_ENABLED=true + LLMSVERIFIER_API_URL=http://llm-verifier:8080 + redeploy`) + sink-side re-validate. The never-degrade guard: wire ONLY after `count(verified)>0`.
- **Option B:** operator seeds the encrypted verifier DB directly (`DATABASE_ENCRYPTION_KEY` is operator-owned).
- **Option C (current):** leave as-is — helixtranslate on the working direct path (NON-degraded). Fully functional; `LLMsVerifier` remains a quality enhancement, not a dependency.

All three are operator-gated because they require the operator's secrets (`config.yaml / DATABASE_ENCRYPTION_KEY`) and/or the external nezha image-build path, neither of which is safely automatable overnight without risking the operator's secret config or a broken verifier image.